



AfriOS

Nouvelle architecture

Vision 6 couches et flux d'exécution

Architecture technique du système AfriOS obtenu après complétion : 6 couches horizontales, 5 runtimes de compatibilité, orchestrateur central unifié, optimisations contexte africain.

VERSION

1.0

DATE

18 August 2026

AUTEUR

Équipe AfriOS

Document technique AfriOS — github.com/ErdisKodjo/System

STATUT

Public

Table des matières

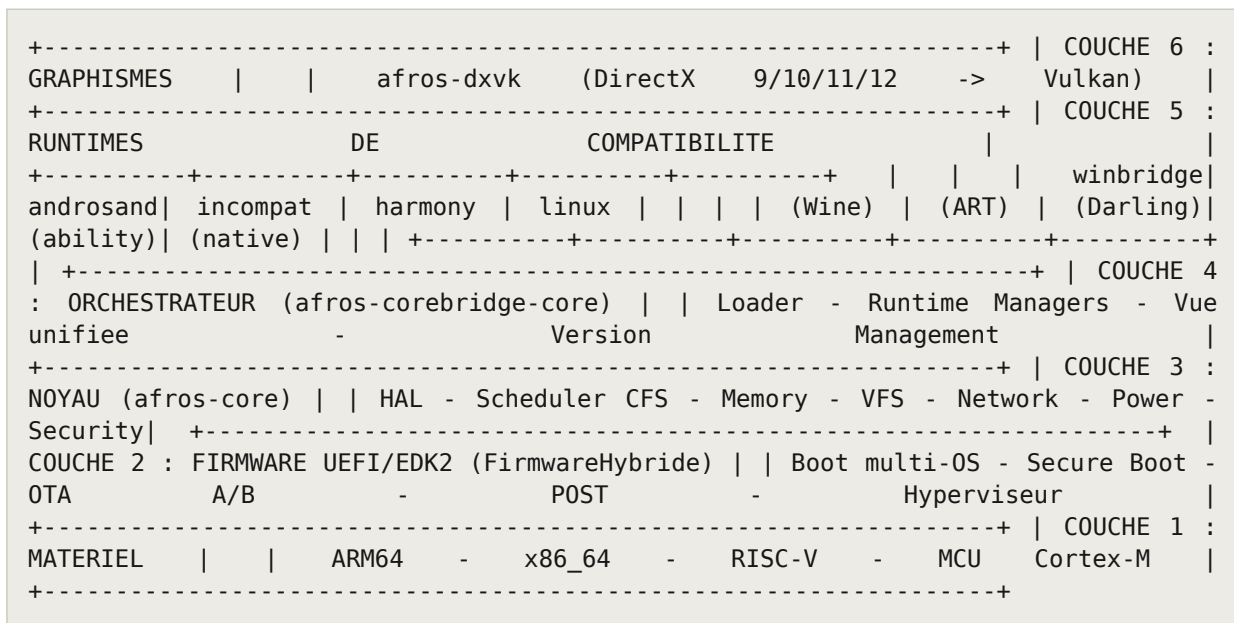
Table des matières	2
1. Vision d'ensemble	4
1.1 Schéma des 6 couches	4
1.2 Principes de design	4
2. Couche 1 — Matériel	5
2.1 Topologie big.LITTLE	5
3. Couche 2 — Firmware UEFI/EDK2	6
3.1 Sequence de boot	6
3.2 Secure Boot et Measured Boot	6
3.3 OTA A/B avec rollback	6
4. Couche 3 — Noyau afros-core	7
4.1 Structure interne du noyau	7
4.2 Ordonnanceur CFS power-aware	7
4.3 Optimisations reseau contexte africain	7
4.4 Gestion d'energie solaire	8
5. Couche 4 — Orchestrateur corebridge	9
5.1 Architecture interne	9
5.2 Loader intelligent	9
5.3 Vue unifiée d'exécution	9
6. Couches 5 & 6 — Runtimes et graphismes	11
6.1 Tableau récapitulatif	11
6.2 Coexistence des runtimes	11
7. Flux d'exécution d'une application	12
7.1 Schéma du flux	12
7.2 Particularités par runtime	12
8. Architecture de sécurité	13
8.1 Chaîne de confiance au boot	13
8.2 Sandboxing des runtimes	13
8.3 Code signing iOS/macOS	13
9. Build system et CI/CD	14
9.1 Organisation du build	14
9.2 CI/CD multi-architectures	14

9.3 Vendorisation des dépendances	14
10. Évolutions futures	15
10.1 Nouveaux ports matériels	15
10.2 Nouveaux runtimes de compatibilité	15
10.3 Optimisations cryptographiques	15
10.4 GPU acceleration réelle	15
10.5 Mode multi-utilisateur	15
11. Conclusion	16

1. Vision d'ensemble

L'architecture finale d'AfriOS s'organise en six couches horizontales, chacune responsable d'un niveau d'abstraction distinct. Cette séparation stricte des préoccupations permet à chaque couche d'évoluer indépendamment — par exemple, ajouter un nouveau port matériel (RISC-V 64) n'impacte que les couches 1 et 2, pas les couches 3 à 6. Réciproquement, ajouter un nouveau runtime de compatibilité (par exemple FreeBSD) n'impacte que les couches 4 et 5, sans toucher au noyau ni au firmware.

1.1 Schéma des 6 couches



1.2 Principes de design

Cinq principes structurels gouvernent l'architecture :

- Separation stricte des preoccupations - chaque couche a un perimetre unique et n'empiete pas sur les responsabilites des autres. Le noyau ne sait pas qu'il existe des apps Windows ; l'orchestrateur ne sait pas comment fonctionne le scheduler CFS.
- Interfaces stables entre couches - chaque couche expose une API formelle (op-table C, vtable C++, ou protocole UEFI) que la couche superieure consomme. Cela permet de remplacer une implementation sans casser les consommateurs.
- Portabilite materielle - le noyau est generique, toutes les decisions architecturales sont dans les ports HAL. Ajouter une architecture = ajouter un port, pas modifier le noyau.
- Mode degrade explicite - chaque module peut signaler AFROS_ERROR_NOT_SUPPORTED au lieu de crasher. L'orchestrateur gere ces retours et propose des alternatives (par exemple : pas de Vulkan -> fallback software rendering).
- Resilience par conception - les modules critiques (DTN, OTA, secure boot) sont concus pour fonctionner en mode degrade : coupure reseau, coupure electrique, corruption disque.

2. Couche 1 — Matériel

AfriOS supporte quatre familles d'architectures matérielles, chacune avec ses propres caractéristiques et cas d'usage. Le choix de ces quatre architectures répond à la diversité du matériel observé sur le terrain africain : du microcontrôleur embarqué au serveur multi-socket.

Architecture	Cœurs	Cas d'usage typique	Exemples hardware
ARM64 (AArch64)	big.LITTLE	Tablettes, SBC, serveurs	Rpi 4, Pine64, Ampere Altra
x86_64	Symmetric	Desktops, laptops, serveurs	Intel/AMD post-2010
RISC-V 64	Symmetric	SBC émergents, IoT	SiFive Hifive, QEMU virt
MCU Cortex-M	Mono-cœur	Microcontrôleurs embarqués	STM32, nRF52, RP2040

Chaque architecture a un port HAL dédié dans `afros-core/Kernel/ports/port-<arch>/` qui implémente les 6 tables d'ops standardisées : `arch_cpu_ops`, `arch_memory_ops`, `arch_interrupt_ops`, `arch_timer_ops`, `arch_console_ops`, `arch_storage_ops`. Un cinquième port `port-host-mock` permet d'exécuter les tests HAL sur un Linux host sans bare-metal.

2.1 Topologie big.LITTLE

ARM64 implémente souvent une topologie big.LITTLE : des cœurs big performants et énergivores, et des cœurs LITTLE plus lents mais économes. AfriOS exploite cette asymétrie via le module `big_little_support.c` qui place les tâches CPU-bound sur big et les tâches IO-bound sur LITTLE, avec hysteresis de 3 échantillons pour éviter le thrashing.

3. Couche 2 — Firmware UEFI/EDK2

Le firmware AfriOS est basé sur EDK2 (TianoCore) et implémente une plateforme package hybride HybridFirmwarePlatformPkg. Il supporte le boot multi-OS (Linux, Windows, macOS, PXE/HTTP), le secure boot, le measured boot (TPM), l'OTA A/B avec rollback, et un hyperviseur minimal pour le passthrough matériel.

3.1 Sequence de boot

```
[1] RESET vector (multi-arch) x86_64: 0xFFFFFFFF0 ARM64: RVBAR_EL1 RISC-V: stvec |  
v [2] SEC phase (Sec/) MultiArchResetVector.S -> Early init, cache, RAM | v [3]  
PEI phase (PlatformInit/Pei/) MemoryInit - PlatformInfoPei (publie HOB FDT/ACPI) |  
v [4] DXE phase (PlatformInit/Dxe/) ACPI table generator - SMBIOS - FDT DXE -  
DeviceTree | v [5] BootManager (BootManager/) BootPolicyEngine -> LinuxBootManager  
/ WinBootMgr / AppleBootHelper / IpxeDriver / HttpBootClient | v [6] AfriOS Kernel  
(kernel_main) HAL init -> Solar power check -> CFS scheduler -> VFS
```

3.2 Secure Boot et Measured Boot

Le secure boot est implémente via SecureBootPolicy.c qui vérifie les signatures des modules UEFI au chargement. Le measured boot (MeasuredBootLib.c) étend les mesures dans un TPM via des PCR (Platform Configuration Registers), permettant à un attester distant de vérifier l'intégrité du boot.

3.3 OTA A/B avec rollback

Le module ABSlotManager.c gère deux slots firmware (A et B) via une variable NVRAM AfriBootInfo contenant :

```
typedef struct { uint8_t active_slot; // 0 = slot A, 1 = slot B uint8_t  
boot_success; // 1 si le boot a réussi uint8_t retry_count; // 0-3, decremente a  
chaque boot uint8_t reserved; } AfriBootInfo_t;
```

Au démarrage, si `boot_success == 0` et `retry_count == 0`, le bootloader bascule sur l'autre slot (rollback automatique). Quand une mise à jour est appliquée via `FwUpdateAgent.c`, elle est écrite sur le slot inactif, et `active_slot` est basculé au prochain reboot avec `retry_count = 3`.

4. Couche 3 — Noyau afros-core

Le noyau AfriOS est un noyau generique multi-architectures structure en sous-systemes. Il ne contient aucune decision architecturale codee en dur - toutes les specificites materielles sont dans les ports HAL. Cette separation permet de porter AfriOS sur une nouvelle architecture en ecrivant uniquement un nouveau port, sans toucher au code noyau.

4.1 Structure interne du noyau

```
afros-core/Kernel/  +--  hal/  Hardware Abstraction Layer | +--  include/
cpu/memory/interrupt/timer/console/storage_abstraction.h | +--  src/  hal_init.c -
power_manager.c   -  device_manager.c   | +--  tests/  hal_smoke_test.c -
hal_test_runner.c +--  ports/ 5 ports HAL (arm64, x86_64, riscv, mcu, host-mock) |
+--  port-<arch>/ 6 fichiers _port.c par arch + CMakeLists.txt +--  afros/ Noyau
generique | +--  scheduler/ afros_cfs.c - big_little.c - power_aware_sched.c | +--
memory/ adaptive_reclaim.c - compression.c - numa_balancer.c | +--  network/
tcp_westwood_africa.c - intermittent_net.c - ROHC | +--  power/ solar_aware.c -
opportunistic_compute.c | +--  fs/afrosfs/ journal.c - file.c | +--  security/
secure_boot.c +--  drivers/ pci/afros_pci.c (extensible) +--  bootmanager/
boot_manager.c (kernel-side boot policy) +--  hypervisor/ hypervisor.h (minimal
hypervisor interface)
```

4.2 Ordonnanceur CFS power-aware

L'ordonnanceur AfriOS est base sur CFS (Completely Fair Scheduler) avec des extensions power-aware. Il gere une file de taches pretes et decide laquelle executer en fonction de la priorite, de la source d'energie (solaire/batterie/AC), et du niveau de batterie. Les taches sont classees en 4 categories :

- CRITICAL : sante, paiement mobile, appels - jamais geles.
- USER_VISIBLE : app foreground - geles a 10% de batterie.
- BACKGROUND : sync, indexing - geles a 25% de batterie.
- BATCH : backups, ML training - jamais acceptees en file si batterie < 50% (sauf mode solaire).

4.3 Optimisations reseau contexte africain

Trois modules reseau implementent des optimisations specifiques au contexte africain :

- TCP Westwood Africa - variante de TCP Westwood+ adaptee aux liens 3G instables. Filtre passe-bas renforce ($\alpha=3$) pour ignorer les pics de handover, detection de lien intermittent ($RTT > 3 \times \text{moyenne} \rightarrow \text{mode survie}$), ssthresh adaptatif pour satellites ($RTT > 400 \text{ ms}$).
- DTN-lite (intermittent_net.c) - couche Delay-Tolerant Networking simplifiee. File en RAM (1024 paquets max) avec priorisation CRITICAL/NORMAL/BULK. Rejeu au retour du lien avec backoff exponentiel (100 ms $\rightarrow$ 5 s max).
- ROHC (protocol_compression.c) - variante de ROHC RFC 3095 pour reduire l'overhead d'en-tetes sur liens 2G. Table de 64 contexts, economie ~40% sur les en-tetes

TCP/IP.

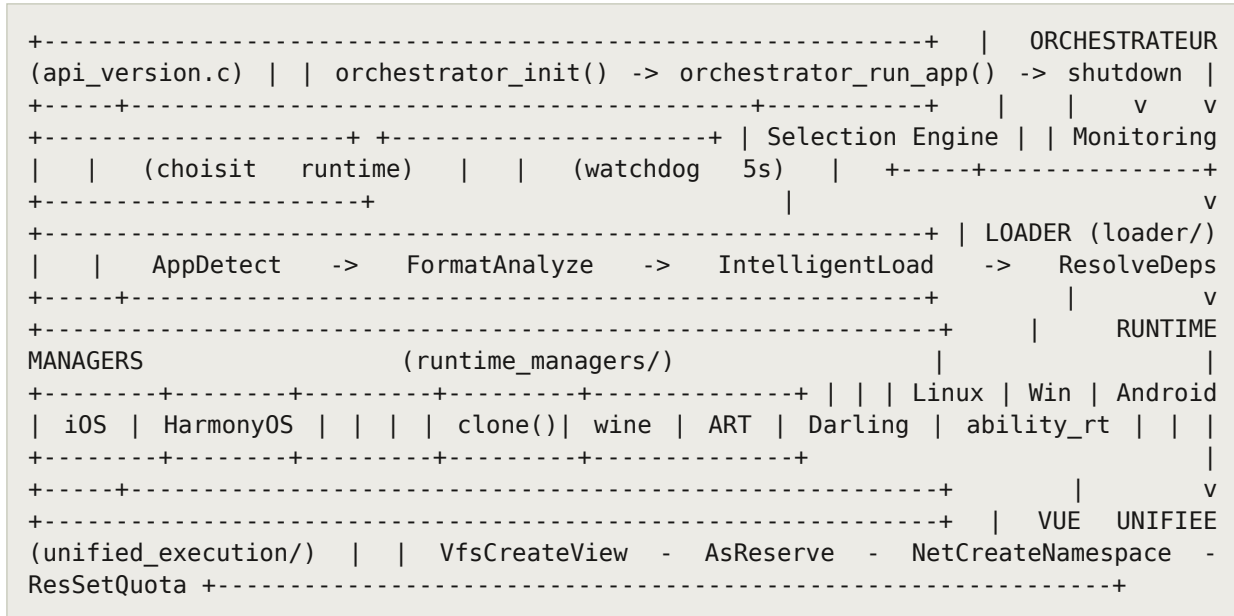
4.4 Gestion d'energie solaire

Quand `afros_hal_ops.get_power_source()` signale `AFROS_POWER_SOURCE_SOLAR`, le module `solar_aware.c` bascule en mode haute performance : frequences CPU maximales, taches BATCH degelées. Le module `opportunistic_compute.c` exploite les fenetres solaires pour executer des taches differées (backups, indexation, ML inference) avec un budget CPU alloue par tache.

5. Couche 4 — Orchestrateur corebridge

L'orchestrateur afros-corebridge-core est le point central de l'AfriOS « plus que complet ». Sans lui, les cinq runtimes de compatibilite sont des ilots isolés ; avec lui, ils deviennent un système unifié où une application Windows peut partager un fichier avec une app Android via une vue VFS commune, ou où une app iOS peut émettre un appel réseau à travers une namespace partagée avec un processus Linux.

5.1 Architecture interne



5.2 Loader intelligent

Le loader est le point d'entree pour toute application lancee sur AfriOS. Il detecte le format du binaire par magic bytes, analyse les details (sous-systeme PE, interpreteur ELF, arch Mach-O, version DEX), consulte le registry des runtimes disponibles, et selectionne le meilleur runtime avec un cache LRU pour les lancements repetes.

Format	Magic bytes	Runtime sélectionné
PE (Windows)	MZ (0x4D5A)	WinRuntimeManager
ELF (Linux)	\x7FELF	LinuxRuntimeManager
Mach-O (iOS/macOS)	0xFEEDFACE / 0xFEEDFACF	IosRuntimeManager
DEX (Android)	dex\n035 (0x6465780A30333500)	AndroidRuntimeManager
HAP (HarmonyOS)	ZIP + .hap extension	HarmonyRuntimeManager

5.3 Vue unifiée d'exécution

Quatre modules fournissent une couche d'abstraction cross-runtime :

- VFS unifie - union view au-dessus de Linux /, Wine Z:/C:, Android /sdcard, iOS sandbox containers, HarmonyOS /data. Traduction de chemins bidirectionnelle.
- Address space partage - registre pthread-protected des regions mmap par runtime, avec partage cross-runtime via AsShare().
- Network namespace - namespaces reseau par runtime avec NAT + port forwarding. Permet a deux runtimes d'utiliser le meme port 8080 sans conflit.
- Resource manager - moniteur de thread 200 ms avec quotas CPU/memoire/IO/FD/ports par runtime. Watchdog tue les runtimes qui ne heartbeat plus pendant 5 s.

6. Couches 5 & 6 — Runtimes et graphismes

Les couches 5 et 6 contiennent les cinq runtimes de compatibilité et la couche de traduction graphique DXVK. Chaque runtime wrappe un backend externe (Wine, ART, Darling, ability_runtime) en exposant une API consommable par l'orchestrateur. Le détail par module est disponible dans le document Modules (document 3/3).

6.1 Tableau récapitulatif

Runtime	Lignes	Backend	Format binaire	API exposée
afros-winbridge	6 160	Wine	PE (.exe/.dll)	Win32/NT syscalls
afros-androsandbox	7 332	ART + Binder	DEX (.apk)	Android Framework
afros-incompat-engine	6 978	Darling	Mach-O (.app)	UIKit/Foundation
afros-harmonygate	5 172	ability_runtime	HAP (.hap)	Ability API
Linux natif	intégré	Linux syscalls	ELF	POSIX
afros-dxvk	5 542	Vulkan	—	D3D9/10/11/12 → Vulkan

6.2 Coexistence des runtimes

Les cinq runtimes peuvent coexister simultanément sur un même système AfriOS. Chaque runtime s'exécute dans son propre sandbox (namespace Linux pour Linux/Windows/Android/HarmonyOS, container iOS pour les apps Darwin). L'orchestrateur applique des quotas par runtime et un watchdog tue les runtimes qui ne heartbeats plus pendant 5 secondes.

Un scénario d'usage typique : un utilisateur peut lancer une app Windows (Notepad++) à côté d'une app Android (WhatsApp) et d'une app iOS (Notes), avec partage de fichiers via la vue VFS unifiée. Les trois apps voient le même /home/user/Documents/ malgré leurs conventions de chemins différentes.

7. Flux d'exécution d'une application

Pour illustrer la collaboration des 6 couches, considérons le scénario suivant : un utilisateur lance un exécutable Windows notepad.exe. Le flux traverse les six couches de bas en haut.

7.1 Schéma du flux

```
[Utilisateur]          ./afros-launch          notepad.exe          |          v
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
orchestrator_run_app(path, args) | | v AppDetect -> magic 'MZ' ->
APP_TYPE_WINDOWS_PE | | v FormatAnalyze -> subsystem=GUI, arch=x86_64 | | v
SelectRuntime -> WinRuntimeManager (compat score) | | v IntelligentLoad -> cache
LRU hit | +-----+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
WinRuntimeManager | | v WinRuntimeSpawn -> démarre wineserver (Unix socket) | | v
wine_loader initialise: registry, filesystem, COM | | v syscall_translator branche
NT -> Linux syscalls | | v PE loader mappe notepad.exe en memoire |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
appel graphique) | | v notepad invoque GDI/Direct2D | | v libafros-dxvk.so
intercepte via alias d3d11.dll.so | | v Traduit D3D11 -> Vulkan | | v Rendu via
Vulkan ICD | +-----+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
(integration silencieuse) | | v Scheduler CFS place le thread wineserver sur big |
| v VFS unifie C:\Users -> /home/user | | v power_aware_sched classe notepad =
USER_VISIBLE | +-----+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
notepad          affichee          a          l'ecran          |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
```

7.2 Particularités par runtime

Le meme flux s'applique aux autres runtimes avec des variations :

- Android : AppDetect voit dex\n035, SelectRuntime choisit AndroidRuntimeManager qui démarre service_manager + surface_flinger + dalvikvm.
- iOS/macOS : AppDetect voit 0xFEEDFACF, IosRuntimeManager démarre dyld_emulator + objc_runtime + macho_loader.
- HarmonyOS : AppDetect voit un ZIP avec extension .hap, HarmonyRuntimeManager démarre ability_runtime daemon et installe le HAP.
- Linux natif : AppDetect voit \x7FELF, LinuxRuntimeManager utilise clone() avec CLONE_NEWNS|CLONE_NEWPID pour sandboxing.

8. Architecture de sécurité

La sécurité est assurée à plusieurs niveaux, depuis le boot jusqu'au runtime applicatif. Cette approche en profondeur (defense in depth) multiplie les barrières qu'un attaquant doit franchir.

8.1 Chaîne de confiance au boot

```
[Hardware Root of Trust] | v (TPM PCR0) [SEC firmware] ---- measured ----> PCR0 | v
(TPM PCR2) [PEI firmware] ---- measured ----> PCR2 | v (TPM PCR4) [DXE drivers]
---- measured ----> PCR4 | v (TPM PCR7) [Secure Boot policy] ---- verifies
signature ----> PCR7 | v (TPM PCR11) [AfriOS kernel] ---- measured ----> PCR11 | v
[Attestation distante possible via TPM Quote]
```

8.2 Sandboxing des runtimes

Chaque runtime s'exécute dans un sandbox dédié :

- Linux/Windows/Android/HarmonyOS : namespace Linux (CLONE_NEWNS|CLONE_NEWPID|CLONE_NEWNET) via clone(). Le runtime ne voit que son namespace.
- iOS/macOS : container dédié ~/Library/Containers/<bundle-id>/ avec entitlements enforcement.
- Quotas : CPU/mémoire/IO/FD/ports par runtime via resource_manager.c. Un runtime ne peut pas épuiser les ressources système.
- Watchdog : tue les runtimes qui ne heartbeats plus pendant 5 s via monitoring.c.

8.3 Code signing iOS/macOS

Le module afros-incompat-engine/codesign/ vérifie les signatures CMS des binaires Mach-O avant exécution : signature_verifier.c valide la signature, certificate_chain.c construit et vérifie la chaîne jusqu'à Apple Root CA, provisioning_profile.c parse les profils .mobileprovision pour extraire l'app ID et le team ID.

9. Build system et CI/CD

Le build system AfriOS est basé sur CMake (>= 3.16) avec une approche target-based moderne. Chaque module déclare ses propres cibles, dépendances et règles d'installation. Le CI/CD GitHub Actions valide automatiquement chaque pull request sur une matrice de 4 architectures.

9.1 Organisation du build

```
CMakeLists.txt (racine) | +-- AFROS_PORT=arm64|x86_64|riscv|mcu|host-mock +--  
AFROS_BUILD_KERNEL=ON +-- AFROS_BUILD_FIRMWARE=ON (necessite EDK2) +--  
AFROS_BUILD_COMPAT=ON +-- AFROS_BUILD_TESTS=ON | v  
add_subdirectory(afros-core/Kernel) -> afros-kernel  
add_subdirectory(afros-core/Kernel/hal) -> afros-hal  
add_subdirectory(afros-core/Kernel/ports/port-${AFROS_PORT}) -> afros-port  
add_subdirectory(afros-corebridge-core) -> afros-corebridge  
add_subdirectory(afros-winbridge) -> afros-winbridge  
add_subdirectory(afros-androsandbox) -> afros-androsandbox  
add_subdirectory(afros-incompat-engine) -> afros-incompat-engine  
add_subdirectory(afros-harmonygate) -> afros-harmonygate  
add_subdirectory(afros-dxvk) -> afros-dxvk add_subdirectory(tests) -> tests  
unitaires
```

9.2 CI/CD multi-architectures

Le workflow ci-workflows/afrios-ci.yml définit 7 jobs GitHub Actions :

- syntax-check - parallèle sur tous les fichiers source, 16 s pour 337 fichiers.
- kernel-build - matrice 4 archs (arm64, x86_64, riscv, mcu) avec cross-compilers dédiés.
- firmware-build - matrice 3 archs (X64, AARCH64, RISC-V) avec EDK2 vendorisé.
continue-on-error: true tant que EDK2 n'est pas vendorisé.
- hal-tests - exécute hal_test_runner sur port-host-mock, 42 tests.
- compat-layer-build - matrice 5 modules de compatibilité.
- docs-build - vérifie que les PDFs se régénèrent.
- worklog-validation - vérifie le format du worklog.

9.3 Vendorisation des dépendances

Le script scripts/fetch-deps.sh clone shallow 9 dépendances externes pinnées dans external/. Les versions sont figées dans le script et documentées dans fetch-deps-versions.md avec licences et cas d'usage. Cette approche garantit la reproductibilité du build sur n'importe quelle machine.

10. Évolutions futures

L'architecture actuelle est concue pour evoluer. Plusieurs axes d'extension sont anticipés et documentés dans le backlog.

10.1 Nouveaux ports matériels

Ajouter une 5e architecture (par exemple LoongArch64 ou Xtensa) se resume a : creer ports/port-<arch>/ avec les 6 fichiers _port.c, un CMakeLists.txt, et un linker script. Le noyau generique n'a pas besoin d'etre modifie. Le CI/CD peut etendre sa matrice en ajoutant une entree.

10.2 Nouveaux runtimes de compatibilité

Ajouter un 6e runtime (par exemple FreeBSD ou Haiku) implique : creer un module afros-<name>bridge/ avec un runtime_manager conforme a l'interface runtime_manager.h, l'enregistrer dans runtime_registry.c, et l'ajouter au detection de l'app_detector.c. L'orchestrateur n'a pas besoin d'etre modifie.

10.3 Optimisations cryptographiques

Plusieurs modules utilisent des stubs cryptographiques (HMAC stand-in dans SoftBus, hash placeholder dans signature_verifier.c). Le branchement d'une vraie HAL crypto AfriOS permettra de remplacer ces stubs par des implementations reelles (CMS RFC 5652, verification de chaine Apple Root CA, HMAC-SHA256).

10.4 GPU acceleration réelle

Le module DXVK compile mais n'emets pas encore les vrais appels vkCmdBind*. Le branchement d'un ICD Vulkan reel (Mesa, NVIDIA) permettra d'activer le rendu hardware-accelerated pour les apps Windows. Le stub actuel suffit pour la validation syntaxique et les tests unitaires.

10.5 Mode multi-utilisateur

Actuellement, AfriOS est single-user. Un mode multi-utilisateur necessiterait d'etendre le VFS unifie avec des per-user namespaces, et d'ajouter un sous-systeme d'authentification (login + session). Ce n'est pas planifie pour v1.x mais est documente comme axe v2.0.

11. Conclusion

L'architecture finale d'AfriOS repose sur six couches horizontales aux responsabilites strictement separees. Cette separation permet une evolution independante de chaque couche et facilite les tests. L'orchestrateur central (afros-corebridge-core) est le coeur fonctionnel : il detecte les formats, selectionne les runtimes, et fournit une vue unifiee du systeme a travers les cinq backends de compatibilite.

Les optimisations specifiques au contexte africain (TCP Westwood Africa, DTN tolerant aux coupures, gestion solaire, ordonnancement power-aware, NUMA balancing, big.LITTLE support) distinguent AfriOS d'une simple distribution Linux generique. Ces choix techniques repondent a des contraintes reelles observees sur le terrain : coupures EDF, handovers cellulaires, deploiements ruraux sur materiel de recuperation.

Le build system CMake unifie et la CI/CD multi-architectures garantissent la reproductibilite et la qualite. Le script `setup.sh --all` valide l'ensemble en 16 secondes sur une machine de dev standard. L'API publique de l'orchestrateur est gelee en v1.0.0 avec un processus RFC documente, ce qui permet aux developpeurs tiers de construire sur AfriOS sans risque de cassure.

Ce document est le deuxieme d'une serie de trois. Le document suivant (Modules) sert de reference par sous-systeme, avec le detail des fichiers, des API exposees, et des dependances pour chacun des modules constitutifs d'AfriOS.